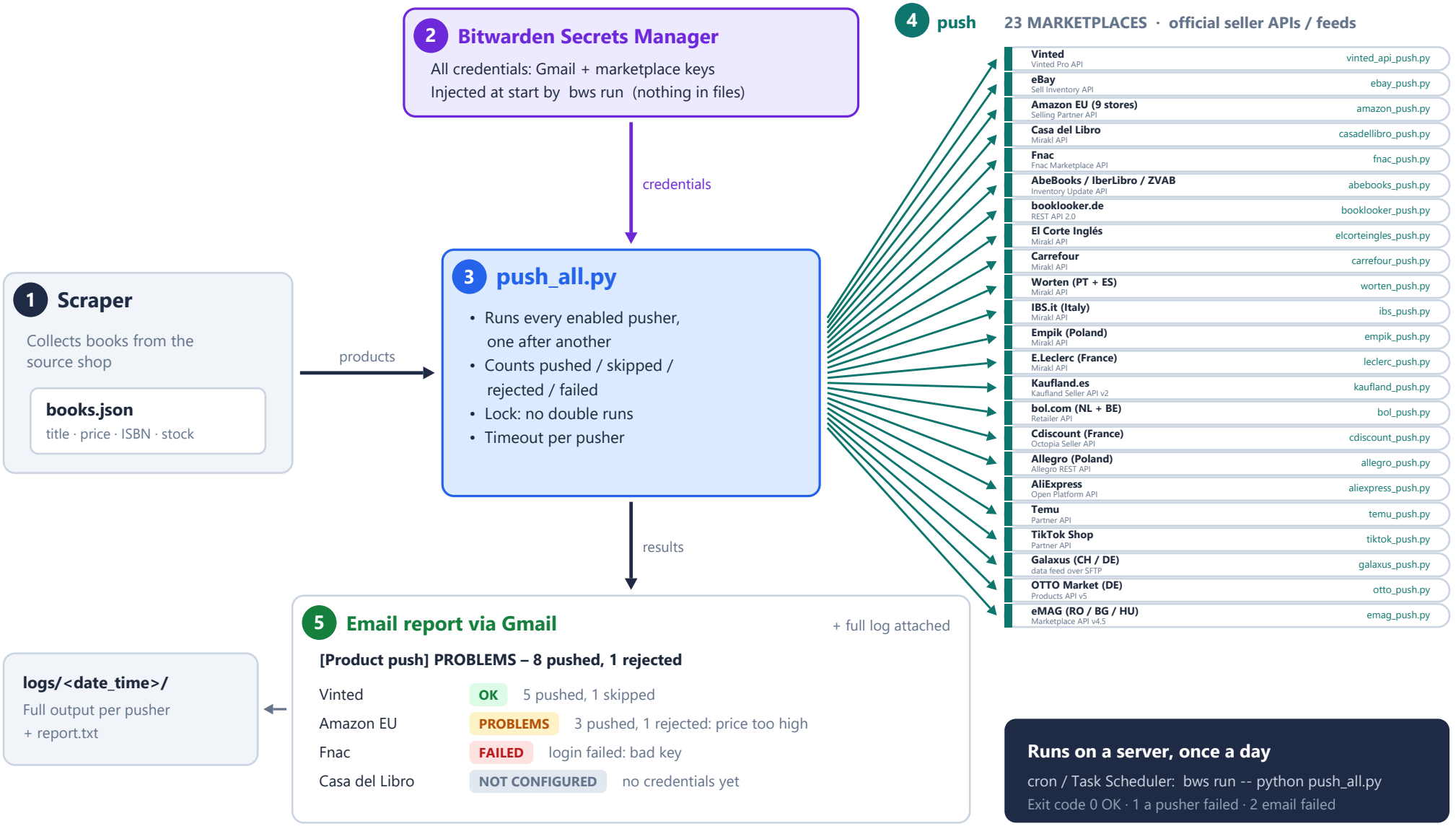
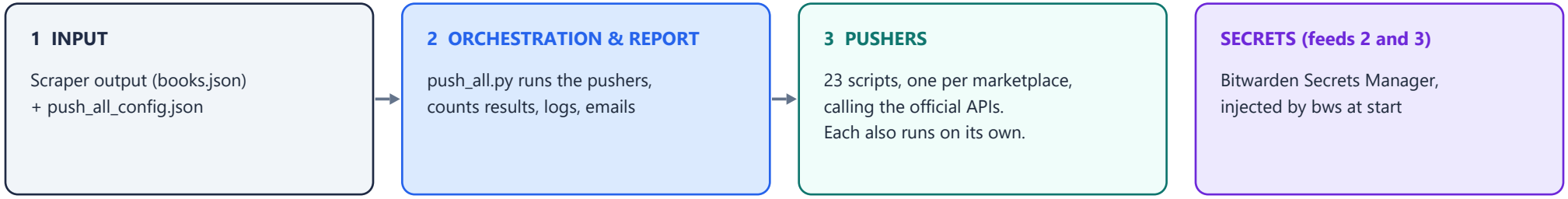
Product pushers: how it works

Scraped books are listed on 23 marketplaces through their official seller APIs and feeds. One run a day; one email tells you what happened everywhere.



Architecture

Four parts, each replaceable on its own. No marketplace logic sits in the runner, and no credentials are stored in code, configuration or git.



Components

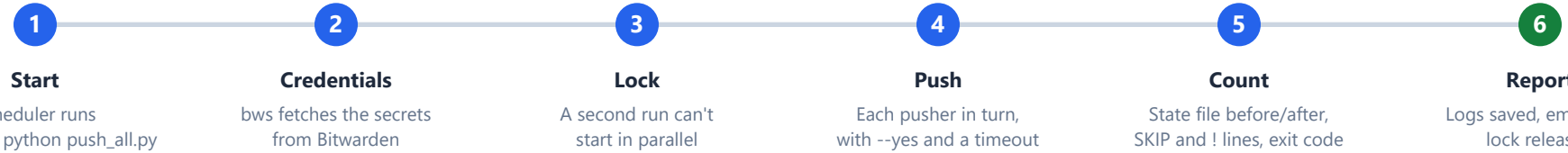
Component	Responsibility	Technology
Scraper (separate project)	Produces books.json: id, title, price, ISBN, condition, images, stock	Python
push_all_config.json	Which pushers run and their options, products file, report recipients, timeouts	JSON (local, not in git)
push_all.py (runner)	Runs the pushers one by one, counts results, writes logs, emails the report, prevents overlapping runs	Python standard library
23 pushers (*_push.py)	Validate products, match the marketplace catalog, create or update listings, record what was pushed	Python + requests (Galaxus: paramiko)
State files (*_posted.json)	What is already listed on each marketplace, so re-runs never create duplicates	JSON (local)
Bitwarden Secrets Manager + bws	Stores all credentials; injects them as environment variables when a run starts	Bitwarden cloud, bws 2.1.0
Gmail	Sends the report: SMTP over TLS, App Password on a dedicated account	Gmail
logs/	Full output of every pusher plus report.txt, one folder per run	Text files (local)

Marketplaces and run flow

Same product format and rules everywhere: skip sold-out or incomplete products, never list a product twice.

Marketplace	API	Login	How a book is listed
Vinted	Vinted Pro Integrations API	Access + signing key (HMAC-signed requests)	Creates a new item with own photos, text and price
eBay (EU)	Sell Inventory API	OAuth refresh token	Inventory item, then offer, then publish
Amazon ES DE FR IT NL BE PL SE IE	Selling Partner API	Login with Amazon refresh token	Finds the Amazon page by ISBN, adds an offer
Casa del Libro	Mirakl seller API	API key	Matches the catalog by EAN; one CSV offer import
Fnac	Fnac Marketplace API v2.6 (XML)	Partner ID + shop ID + key	Sends offers by EAN in batches; checks the batch status
AbeBooks / IberLibro (+ ZVAB)	Inventory Update API (XML)	Username + API key	Describes its own copy of a book: no catalog match needed
booklooker.de (Germany)	booklooker REST API 2.0	API key	Own copy of a book; all offers in one article file
El Corte Inglés	Mirakl seller API	API key	Same as Casa del Libro (no public docs)
Carrefour	Mirakl seller API	API key	Same as Casa del Libro (ES and FR are separate)
Worten (Portugal + Spain)	Mirakl seller API	API key	Same as Casa del Libro; one price for PT + ES
IBS.it (Italy)	Mirakl seller API	API key	Same as Casa del Libro
Empik (Poland)	Mirakl seller API	API key	Same as Casa del Libro; prices converted to PLN
E.Leclerc (France)	Mirakl seller API	API key	Same as Casa del Libro
Kaufland.es	Kaufland Seller API v2	Client + secret key (HMAC-signed)	Checks the catalog by EAN, then adds an offer
bol.com (NL + BE)	Retailer API (Offers v11)	Client ID + secret	Catalog check by EAN, then an offer (new or used)
Cdiscount (France)	Octopia Seller API	Client ID + secret + seller ID	Catalog check by EAN, offers sent in Upsert packages
Allegro (Poland)	Allegro REST API	Client ID + secret + device authorization	Offer on the catalog product by EAN; prices in PLN
AliExpress (local seller)	AliExpress Open Platform API	App key + secret + OAuth seller token	Own product per book (category from EAN data)
Temu (local seller)	Temu Partner API	App key + secret + Seller Center token	Own product per book; Temu reviews it and its price
TikTok Shop (Spain, EU)	TikTok Shop Partner API	App key + secret + OAuth seller token	Own product per book; TikTok reviews it
Galaxus (CH + DE)	Data feed (CSV) over SFTP	SFTP user + password (pinned host key)	One feed file with all offers, matched by ISBN
OTTO Market (Germany)	OTTO Market API (Products v5)	Client ID + secret (self-app)	Own product per book; stock via Availability API
eMAG (RO, BG, HU)	eMAG Marketplace API v4.5	API user + password (whitelisted IP)	Finds the eMAG product by EAN, attaches the offer

One run, step by step



Security, operations and status

What protects the credentials, how failures are handled, and where the project stands today.

Credentials

Decision	Why
All keys in **Bitwarden Secrets Manager**	One managed place, team access, access log; change a key without touching the server
Server holds only a **read-only access token**	Smallest secret on disk; each machine can be revoked separately
Keys reach scripts only as **environment variables**	Nothing secret in code, git or config files
Gmail **App Password** on a dedicated account	Revocable on its own; no personal mailbox exposed
Email only over **TLS**; local files in .gitignore	Passwords never sent in clear text; nothing local reaches GitHub

When something goes wrong

Situation	What happens
Product invalid	Skipped with its reason; the rest continue
Offer rejected	Shown in the email with the marketplace's message; retried next run
Bad key / crash / hang	That marketplace is FAILED; the others still run
Email can't be sent	Exit code 2; the report is still in logs/

Status (5 Nov 2026)

Part	Status	Details
23 pushers	TESTED	Pass all automated tests against fake APIs. Not yet run on the real marketplaces: no seller accounts yet.
Runner + email	VERIFIED	Real test email and report delivered through Gmail, with credentials from Bitwarden.
Bitwarden	VERIFIED	Project, machine account, token and bws working.
Server	TO DO	Steps and schedule commands are documented.

Automated tests: **506 checks, all passing** (24 test suites, no real API contacted).

Next steps

1. Seller accounts and API access for each marketplace
2. Add their keys to Bitwarden
3. Safe first run per marketplace: dry run, then 1 product
4. Deploy to the server and schedule daily

Note: new books in Spain, Portugal, Italy, France, Germany and the Netherlands have legally fixed prices.
Full documentation: docs/architecture.md and docs/push_all.md in the GitHub repository.